

使用 Kiro 操作 Amazon SageMaker

完整實作手冊

2026 年 8 月

使用 Kiro 操作 Amazon SageMaker 完整實作手冊

適用對象：黑客松參與者（不限技術背景） 環境：AWS Workshop Studio 提供之獨立帳號（Region：us-west-2） 預期成果：透過與 Kiro 對話，於 SageMaker 雲端完成一次傳統機器學習模型（XGBoost）的訓練，並選擇性地將模型部署為線上推論服務。 版本說明：本手冊之所有步驟均已實際執行驗證通過（Iris 分類，最終推論結果為 Iris-virginica）。

如何閱讀本手冊：凡是以灰底方框呈現的區塊，就是要你原封不動複製、貼給 **Kiro** 的指示（可用滑鼠框選複製，或直接照著唸）。方框以外的文字是說明與背景，供你理解，不需複製。方框內若出現以角括號標示的佔位符（例如 <貼上你的 S3 路徑>），請替換成你自己的實際內容後再送出。

目錄

1. 總覽與學習目標
 2. 事前準備
 3. 核心概念（三分鐘理解）
 4. Step 1：連上你的 AWS 帳號
 5. Step 2：建立 Python 執行環境
 6. Step 3：產生訓練資料並上傳 S3
 7. Step 4：送出訓練任務
 8. Step 5：確認訓練成果
 9. Step 6（進階）：部署模型並呼叫推論
 10. Step 7：資源清理
 11. 常見問題與排解
 12. 附錄：與 Kiro 對話的技巧
-

1. 總覽與學習目標

本手冊帶你完成一件事：不在自己的電腦上訓練模型，而是透過 **Kiro** 指揮 **Amazon SageMaker**，在雲端完成訓練，再把成果取回。

完成本手冊後，你將能夠：

- 讓本機的 Kiro 連上一個 AWS 帳號並代為操作。
- 以自然語言請 Kiro 產生訓練資料、建立儲存空間並上傳。
- 送出一個 SageMaker 訓練任務，並在主控台觀察其由 InProgress 轉為 Completed。
- （進階）將訓練完成的模型部署為即時推論端點（Endpoint），並實際呼叫取得預測結果。
- 正確清理資源，避免產生非預期費用。

核心體驗：整個過程你不需要手寫程式。你只需描述目標，由 Kiro 產生程式碼、執行並協助除錯。

2. 事前準備

開始前請確認下列項目：

項目	說明
Workshop Studio 活動連結	由講師提供，用於登入你的專屬 AWS 帳號
已安裝並可登入的 Kiro	於本機執行
終端機（Terminal）操作權限	Kiro 需透過終端機執行程式
網路連線	僅需一般網路即可，訓練運算在雲端執行

本手冊僅需在本機安裝一個套件（boto3），其餘機器學習相關套件皆於雲端容器內執行，無需在本機安裝。

3. 核心概念（三分鐘理解）

在動手前，先理解三個關鍵概念，有助於你理解每一步在做什麼。

3.1 為什麼訓練不在你的電腦上進行 SageMaker 的訓練工作（Training Job）是在 AWS 託管的運算容器中執行。你只需送出「設定」——包含資料位置、演算法、機器規格與超參數——AWS 便

會自動啟動一台機器執行訓練，完成後自動關閉，並將訓練好的模型存回 Amazon S3。因此你的本機不需安裝任何機器學習套件，也不會佔用本機資源。

3.2 為什麼本機只需要 boto3 boto3 是 AWS 的 Python SDK，體積輕巧。它負責「發送指令」給 AWS（例如：建立訓練任務、建立端點）。真正吃資源的訓練與推論，都在雲端容器內完成。這也是本手冊刻意不使用較龐大的 sagemaker SDK 的原因——避免現場因套件相依而卡住。

3.3 三個你會在主控台看到的產物

產物	位置	意義
Training Job	SageMaker → Training → Training jobs	一次訓練任務，狀態由 InProgress 轉為 Completed
模型檔 (model.tar.gz)	Amazon S3	訓練完成後產出的模型檔案
Endpoint	SageMaker → Inference → Endpoints	(進階) 將模型部署為可即時呼叫的線上服務

4. Step 1: 連上你的 AWS 帳號

你的 Kiro 執行於本機，但要操作的是 Workshop Studio 配發的雲端帳號。因此第一步必須將該帳號的臨時憑證提供給本機，Kiro 才有權限代為操作。

4.1 取得臨時憑證

1. 開啟講師提供的 Workshop Studio 活動連結，以個人 email 登入。
2. 於頁面中找到「Get AWS CLI credentials」（或標示為 AWS CLI 的區塊）。
3. 複製該處提供的憑證，內容包含 Access Key ID、Secret Access Key 與 Session Token。
4. 記下畫面顯示的 Region，本場為 **us-west-2**。

4.2 將憑證提供給本機（通用做法，不分環境）

不論你的電腦是否安裝 AWS CLI、是否使用 pyenv 等工具，下列做法皆適用。你不需要自行判斷環境，也不需要執行 `aws configure`。只要將完整的憑證內容連同下列指示一併交給 Kiro，由它完成設定與驗證：

以下是我從 Workshop Studio 取得的 AWS 臨時憑證。請幫我完成設定，讓我在這台電腦上執行的 Python (boto3) 以及你 (Kiro) 都能直接使用，過程中我不需要指定任何 profile 名稱：

1. 請將這組憑證 (Access Key ID、Secret Access Key、Session Token) 寫入 `~/.aws/credentials`，放在 `[default]` 這個 profile 底下。

2. 請將 `region` 設定為 `us-west-2`，寫入 `~/.aws/config` 的 `[default]` profile。
3. 設定完成後，請用 `boto3` 呼叫 STS 的 `get_caller_identity` 進行驗證（不要使用 `aws` 指令，因為我的環境可能未安裝 AWS CLI，或該指令會被攔截），並印出回傳的帳號 ID。

憑證如下：

（在此貼上從 Workshop Studio 複製的整段憑證）

這個做法之所以在任何環境都可行，原因有三：

- 寫入設定檔，而非環境變數：設定會持續存在，不會因為換一個終端機視窗、或 Kiro 另開一個工作階段而失效。
- 使用 **default profile**：後續所有步驟的程式都無需特別指定 profile，`boto3` 會自動採用，可避免「憑證已設定卻讀不到」的情形。
- 以 **boto3** 驗證，而非 **aws** 指令：完全繞開「未安裝 AWS CLI」與「`pyenv` 攔截 `aws` 指令」這兩種常見狀況。

本步驟檢核點：Kiro 透過 `boto3` 成功印出你的帳號 ID。（若你的電腦確實安裝了可用的 AWS CLI，亦可自行執行 `aws sts get-caller-identity` 再次確認，但此非必要。）

注意：臨時憑證具有有效期（通常為數小時）。若後續操作中出現「找不到憑證」的錯誤，代表憑證已過期，回到 Workshop Studio 頁面重新複製一次即可。

5. Step 2: 建立 Python 執行環境

本步驟的目標，是為後續程式準備一個乾淨、獨立的 Python 環境（稱為虛擬環境，`venv`）。使用虛擬環境有兩個好處：其一，套件安裝在專屬環境內，不會與系統原有的 Python 衝突；其二，可避免部分系統對直接安裝套件的限制（例如某些 macOS 環境會將系統 Python 標記為「externally-managed」而阻擋 `pip install`，此為實測遇到的情形）。

虛擬環境是 Python 內建、跨平台皆適用的機制，Windows、macOS 與 Linux 都支援。你不需要自行記憶各作業系統的指令差異——只要將目標交給 Kiro，由它依你的作業系統採用正確的建立與啟用方式。可將下列指示提供給 Kiro：

請在目前資料夾建立一個 Python 虛擬環境，命名為 `.venv`。請依我的作業系統採用正確的建立與啟用指令（Windows 與 macOS/Linux 的啟用方式不同，請你自行判斷）。啟用後，在其中安裝 `boto3` 與 `scikit-learn`。之後所有程式都請以這個虛擬環境執行。

Kiro 會偵測你的作業系統、建立 `.venv`、以對應的方式啟用並完成套件安裝，後續執行程式時亦會沿用此環境。你無需關心底層指令差異。

參考說明：不同作業系統啟用虛擬環境的指令略有不同，Kiro 會自動處理，此處僅供理解：

作業系統	啟用指令 (Kiro 會自動採用)
Windows (PowerShell)	<code>.venv\Scripts\Activate.ps1</code>
Windows (命令提示字元 cmd)	<code>.venv\Scripts\activate.bat</code>
macOS / Linux	<code>source .venv/bin/activate</code>

本步驟檢核點：`.venv` 已建立，且 `boto3` 已安裝完成（可請 Kiro 於該環境中執行 `python -c "import boto3"` 確認無誤）。

6. Step 3: 產生訓練資料並上傳 S3

傳統機器學習需要資料。本手冊使用經典的鳶尾花 (Iris) 分類資料集作為範例。你不需自行整理格式——請 Kiro 產生符合 SageMaker XGBoost 要求的資料並上傳。可將下列指示提供給 Kiro：

我的帳號目前是空的，region 為 us-west-2。我要使用 SageMaker 內建的 XGBoost 演算法建立一個表格分類模型。請協助我：

1. 使用 `scikit-learn` 的 Iris 資料集產生約 120 筆訓練資料。
2. 寫成符合 SageMaker XGBoost 要求的 CSV 格式：不含表頭、第一欄為 label (數字)，存成 `train.csv`。
3. 使用 `boto3` 建立一個新的 S3 bucket (名稱取易於辨識者，例如 `kiro-workshop-你的名字`)。請注意 us-west-2 建立 bucket 需指定 `LocationConstraint`。
4. 將 `train.csv` 上傳至該 bucket。
5. 印出資料在 S3 的完整路徑 (`s3://...`)。完成後直接執行。

重點說明：SageMaker XGBoost 對輸入格式有明確要求——CSV 不含表頭，且第一欄必須為 label。這部分由 Kiro 處理，你無需手動調整。

本步驟檢核點：Kiro 印出 S3 路徑（例如 `s3://kiro-workshop-你的名字/.../train.csv`）；於 S3 主控台可看到該檔案。請記下此路徑，下一步會用到。

7. Step 4: 送出訓練任務

這是整個流程的核心。你將請 Kiro 送出一個 SageMaker 訓練任務。

7.1 關於執行角色 (重要)

SageMaker 執行訓練時，需要一個「執行角色（IAM Role）」，代表 SageMaker 去讀取 S3 資料與啟動運算資源。經實測，Workshop Studio 配發的帳號中並未預先建立可用的 SageMaker 執行角色，僅有系統的 service-linked role（此類角色不能作為訓練任務的 RoleArn）。因此，下列指示已納入「自動尋找，若無則自動建立」的邏輯，由 Kiro 一併處理。

7.2 送出訓練任務

將下列指示提供給 Kiro：

```
請使用 boto3（不要使用 sagemaker SDK），於 .venv 中撰寫一個 train.py，呼叫 SageMaker 的 create_training_job，使用內建的 XGBoost 演算法訓練我剛才上傳的 train.csv。
```

關於 execution role，請依下列邏輯處理：

1. 使用 boto3 列出帳號中的 IAM roles，尋找可用的 SageMaker 執行角色。
2. 必須過濾掉路徑含有 aws-service-role 的角色（此為 service-linked role，不能作為訓練任務的 RoleArn）。
3. 若找不到可用角色，請自動建立一個：名稱為 kiro-sagemaker-execution-role，信任政策允許 sagemaker.amazonaws.com 擔任，附加 AmazonSageMakerFullAccess 與 AmazonS3FullAccess 政策，建立後等待約 10 秒讓 IAM 生效再使用。

其他設定：

- 訓練資料 S3 路徑：<貼上 Step 3 印出的 S3 路徑>，input channel 名稱使用 train。
 - 自動偵測 region (us-west-2) 並取得內建 XGBoost 容器 image URI。取用時請指定明確的版本號（例如 1.7-1），不要使用 :latest 或 :1 作為標籤。
 - 訓練 instance 使用 ml.m5.xlarge，一台。
 - 設定基本 XGBoost 超參數（objective 使用 multi:softmax、num_class 為 3、num_round 使用合理值，例如 100）。
 - 訓練任務命名為 kiro-xgb-<你的名字>-<時間>。
 - 模型輸出至我的 S3。
 - 最後印出訓練任務名稱。
- 完成後直接執行 train.py。

7.3 等待訓練完成

訓練於雲端執行，通常需數分鐘（含機器啟動時間）。可請 Kiro 協助輪詢狀態：

```
請於 .venv 中撰寫一小段程式，使用 boto3 每 30 查詢一次剛才訓練任務的狀態，當狀態變為 Completed 或 Failed 時停止，並印出最終狀態。
```

你也可以同時開啟 SageMaker 主控台 → Training → Training jobs，觀察任務狀態變化。

本步驟檢核點：主控台中該訓練任務狀態由 InProgress 轉為 **Completed**。

8. Step 5: 確認訓練成果

訓練完成後，確認模型產物與訓練指標。將下列指示提供給 Kiro：

我的訓練任務已完成。請使用 `boto3` 印出：

1. 訓練完成的模型檔 (`model.tar.gz`) 在 S3 的完整位置。
2. 此訓練任務的最終訓練指標 (若有)。

至此，你已透過與 Kiro 對話，在雲端完成一次完整的模型訓練。建議於主控台的 Training jobs 頁面擷取畫面，作為完成紀錄。

本步驟檢核點：取得 `model.tar.gz` 的 S3 位置；主控台顯示 Completed。

9. Step 6 (進階)：部署模型並呼叫推論

此步驟為進階選項。完成核心訓練後，若時間充裕，可進一步將模型部署為即時推論端點並實際呼叫。

9.1 部署為端點

請使用 `boto3` 將我剛才訓練完成的 XGBoost 模型 (`model.tar.gz` 位於 S3) 部署為一個 real-time endpoint。步驟為 `create_model`、`create_endpoint_config`、`create_endpoint`。

- 自動取得最近一次訓練任務的 model artifact 路徑與其使用的 role ARN。
- 使用相同的 XGBoost 容器 image URI (推論亦使用此容器)。
- 推論 instance 使用 `ml.m5.xlarge`，一台。
- endpoint 命名為 `kiro-xgb-endpoint-<你的名字>`。
- 使用 `waiter` 等待 endpoint 變為 `InService` (約 5 至 8 分鐘)，並印出 endpoint 名稱。

撰寫成 `deploy.py` 並執行。

9.2 呼叫推論

請撰寫 `invoke.py`，使用 `boto3` 呼叫我剛部署的 endpoint (`kiro-xgb-endpoint-<你的名字>`)。送入一筆測試特徵 (四個數字，Iris 格式，CSV 一行，不含 label)，呼叫時 `ContentType` 請設定為 `text/csv` (XGBoost endpoint 僅接受 `text/csv`)。並將回傳的類別對照 Iris 類別名稱 (0 為 `setosa`、1 為 `versicolor`、2 為 `virginica`) 後印出。完成後直接執行。

本步驟檢核點：主控台中 endpoint 狀態為 `InService`；終端機印出推論結果 (例如 2 對應 Iris-`virginica`)。

10. Step 7：資源清理

- 訓練任務（**Training Job**）：訓練完成後機器已自動關閉，不會持續計費，無需刪除。
- 推論端點（**Endpoint**）：若你完成了 Step 6，端點會持續在線並計費，務必刪除。

將下列指示提供給 Kiro：

請使用 `boto3` 刪除我剛才建立的 SageMaker endpoint (`kiro-xgb-endpoint-<你的名字>`)，連同其 `endpoint config` 與 `model` 一併清除，並確認皆已刪除。

Workshop Studio 活動結束後通常會自動回收整個帳號，但當場刪除端點仍為良好習慣。

本步驟檢核點：Endpoints 頁面已無你建立的端點。

11. 常見問題與排解

下表整理實測過程中遇到的問題與對應解法。遇到任何錯誤時，最有效的做法是將完整錯誤訊息貼給 Kiro，請它協助修正：

我執行時出現以下錯誤，請協助判斷原因並修正，然後重新執行：
(貼上終端機的完整錯誤訊息)

錯誤訊息	原因	解法
Unable to locate credentials	憑證未設定或已過期	回 Workshop Studio 重新取得憑證；或請 Kiro 將憑證寫入 <code>~/.aws/credentials</code>
ModuleNotFoundError: No module named 'boto3'	未在虛擬環境中執行	請 Kiro 於 <code>.venv</code> 中安裝 <code>boto3</code> 並以該環境執行
externally-managed-environment	系統 Python 阻擋直接安裝套件 (常見於部分 macOS/ Linux 環境)	建立並使用虛擬環境 (<code>.venv</code>)，詳見 Step 2
找不到可用的 execution role / AccessDenied	帳號無可用角色，或誤用 service-linked role	過濾路徑含 <code>aws-service-role</code> 的角色；若無可用角色則自動建立
ValidationException: 找不到 image	XGBoost 容器 image URI 有誤	請 Kiro 依 <code>us-west-2</code> 重新取得內建 XGBoost 的 image URI，並指定明確版本號 (例如 <code>1.7-1</code>)，不要用 <code>:latest</code> 或 <code>:1</code>
推論回傳格式錯誤 / 無法解析	呼叫 endpoint 未指定正確的內容類型	XGBoost endpoint 僅接受 <code>text/csv</code> ，請 Kiro 於呼叫時將 <code>ContentType</code> 設為 <code>text/csv</code>
SoftmaxMultiClassObj: label must be in [0, num_class)	label 值超出範圍，或 num_class 設定不符	

錯誤訊息	原因	解法
		確認 label 為從 0 起算的整數，且 num_class 等於實際類別數（Iris 為 3）

排解技巧：

- 貼錯誤訊息時請完整複製，不要只貼一行。
- 若修正後仍有錯誤，將新的錯誤訊息再次貼給 Kiro，它會逐步逼近正確解法。
- 屬於權限層級的問題（例如帳號本身缺乏建立角色的權限），請向現場助教尋求協助。

12. 附錄：與 Kiro 對話的技巧

- 描述目標，而非撰寫程式：你只需說明想達成什麼，Kiro 會產生對應的程式碼並執行。
- 一次一個目標：將任務拆成清楚的小步驟（產生資料、送訓練、部署、呼叫），比一次要求全部更穩定。
- 善用錯誤訊息：出錯是正常的。將錯誤原封不動貼回給 Kiro，是最快的除錯方式，也最能體現 Kiro 的協作價值。
- 要求它沿用既有環境：明確告訴 Kiro「使用 .venv 執行」，可避免落回被系統阻擋的預設 Python。
- 完成後要求確認：例如請 Kiro 印出資源名稱、狀態或路徑，便於你在主控台核對。

流程速覽

步驟	你對 Kiro 的指示重點	完成標準
Step 1 連帳號	寫入臨時憑證、設定 region	get-caller-identity 成功
Step 2 建環境	建立 .venv、安裝 boto3	套件安裝完成
Step 3 備資料	產生 Iris 資料、建 bucket、上傳	取得 S3 路徑
Step 4 送訓練	自動處理角色、送出 XGBoost 訓練	任務 Completed
Step 5 確認成果	印出模型位置與指標	取得 model.tar.gz
Step 6 部署推論（進階）	部署 endpoint、呼叫推論	印出預測結果
Step 7 清理	刪除 endpoint 與相關資源	資源已清除